Lab 7 — Xác suất

1. Đề bài tiếng Anh

LAB SESSION 7

Problem Description

Probability, Simulation, Conditional Probability, Bayes' Theorem and Probability Distributions.

Requirements

Task 1 – Sample Space and Basic Probability

A box contains 10 cards numbered from 1 to 10. One card is randomly selected.

Define the sample space and events:

- A: The selected number is even.
- B: The selected number is greater than 6.

Calculate theoretical probabilities `P(A)`, `P(B)`, `P(A ∩ B)`, and `P(A ∪ B)`.

Task 2 – Probability Simulation

A basketball player has a 70% probability of successfully making a free throw.

- Use NumPy to simulate **5,000 free throws** and calculate experimental probability of success.
- Simulate two independent players with success probabilities 0.7 and 0.6.
- Calculate probability both succeed and probability at least one succeeds.
- Compare simulation with theoretical probabilities.

Task 3 – Conditional Probability

A day is randomly selected from a seven-day week.

- Event A: The selected day is a weekday (Monday to Friday).
- Event B: The selected day is Monday, Tuesday, Wednesday or Thursday.

Calculate theoretical conditional probability `P(A | B)`. Simulate 5,000 selections using Python and NumPy, then compare the experimental result.

Task 4 – Bayes' Theorem

An email filtering system has these characteristics:

- 30% of emails are spam.
- 90% of spam emails are correctly identified as spam.

- 5% of non-spam emails are incorrectly classified as spam.

Calculate overall probability an email is classified as spam, then use Bayes' theorem to calculate probability an email is actually spam when classified as spam.

Task 5 – Probability Distributions

A customer service team receives 8 independent requests. Each has a 75% probability of successful resolution.

- Use `scipy.stats.binom.pmf()` to calculate probability of **exactly 6** resolved requests and **at least 6** requests.
- Plot the Binomial probability distribution using Matplotlib.

Chuyển từ `LAB SESSION 7.docx` sang Markdown.

2. Đề bài tiếng Việt

LAB SESSION 7 — Xác suất

Một chuỗi bài thực hành về **xác suất cơ bản, mô phỏng, xác suất có điều kiện, định lý Bayes và phân phối xác suất**.

Toàn bộ yêu cầu

Bài 1

Một hộp có 10 thẻ đánh số từ 1 đến 10, lấy ngẫu nhiên một thẻ. Xác định không gian mẫu, biến cố A (số chẵn), B (số lớn hơn 6). Tính xác suất của A, B, đồng thời A và B, và ít nhất A hoặc B.

Bài 2

Một người chơi bóng rổ ném phạt thành công với xác suất 70%. Dùng NumPy (thư viện tính toán mảng) mô phỏng **5.000** lần để tính tỉ lệ thực nghiệm. Tiếp theo mô phỏng hai người độc lập với xác suất thành công 0,7 và 0,6; tính cả hai thành công và ít nhất một người thành công; so sánh với lý thuyết.

Bài 3

Chọn ngẫu nhiên một ngày trong tuần gồm 7 ngày. A là ngày thường từ thứ Hai đến thứ Sáu, B là thứ Hai đến thứ Năm. Tính `P(A|B)`, mô phỏng 5.000 lần bằng Python/NumPy (thư viện tính toán mảng) và so sánh với kết quả lý thuyết.

Bài 4

Hệ thống lọc email có 30% email spam, phát hiện đúng 90% spam, và báo nhầm 5% email không spam thành spam. Tính xác suất một email được đánh dấu spam và xác suất email thực sự spam khi đã bị đánh dấu.

Bài 5

Nhóm hỗ trợ nhận 8 yêu cầu độc lập, mỗi yêu cầu có khả năng giải quyết thành công 75%. Dùng `scipy.stats.binom.pmf()` tính xác suất giải quyết **đúng 6** và **ít nhất 6**, vẽ phân phối nhị thức bằng Matplotlib (thư viện vẽ đồ thị).

3. Code Python có chú thích

```
from pathlib import Path  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.
import sys  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.
import numpy as np  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.
from scipy.stats import binom  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.
```

```
SEED=2026  # Tính hoặc cập nhật biến SEED theo công thức của bài.
TRIALS=5000  # Tính hoặc cập nhật biến TRIALS theo công thức của bài.

def task01_cards():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    space=set(range(1,11))  # Tính hoặc cập nhật biến space theo công thức của bài.
    a={x for x in space if x%2==0}  # Tính hoặc cập nhật biến a theo công thức của bài.
    b={x for x in space if x>6}  # Tính hoặc cập nhật biến b theo công thức của bài.
    return {  # Trả kết quả (return: trả về) cho nơi gọi hàm.
        "space":sorted(space),"A":sorted(a),"B":sorted(b),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(A)":len(a)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(B)":len(b)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(A and B)":len(a&b)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(A or B)":len(a|b)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
    }  # Đóng danh sách, mảng hoặc nhóm tham số đang xây dựng.

def task02_free_throws():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    rng=np.random.default_rng(SEED)  # Tạo hoặc sử dụng số ngẫu nhiên; seed giúp lặp lại phép thử.
    single=rng.random(TRIALS)<0.7  # Tạo hoặc sử dụng số ngẫu nhiên; seed giúp lặp lại phép thử.
    first=rng.random(TRIALS)<0.7  # Tạo hoặc sử dụng số ngẫu nhiên; seed giúp lặp lại phép thử.
    second=rng.random(TRIALS)<0.6  # Tạo hoặc sử dụng số ngẫu nhiên; seed giúp lặp lại phép thử.
    both=first & second  # Tính hoặc cập nhật biến both theo công thức của bài.
    either=first | second  # Tính hoặc cập nhật biến either theo công thức của bài.
    return {  # Trả kết quả (return: trả về) cho nơi gọi hàm.
        "experimental_single":single.mean(),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "experimental_both":both.mean(),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "experimental_either":either.mean(),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "theoretical_single":0.7,  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "theoretical_both":0.7*0.6,  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "theoretical_either":1-(1-0.7)*(1-0.6),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
    }  # Đóng danh sách, mảng hoặc nhóm tham số đang xây dựng.

def task03_conditional_days():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    A=set(range(5))  # Tính hoặc cập nhật biến A theo công thức của bài.
    B=set(range(4))  # Tính hoặc cập nhật biến B theo công thức của bài.
    theoretical=len(A&B)/len(B)  # Tính hoặc cập nhật biến theoretical theo công thức của bài.
    rng=np.random.default_rng(SEED+1)  # Tạo hoặc sử dụng số ngẫu nhiên; seed giúp lặp lại phép thử.
    drawn=rng.integers(0,7,size=TRIALS)  # Tạo hoặc sử dụng số ngẫu nhiên; seed giúp lặp lại phép thử.
    given_b=np.isin(drawn,list(B))  # Tính hoặc cập nhật biến given_b theo công thức của bài.
    joint=np.isin(drawn,list(A&B))  # Tính hoặc cập nhật biến joint theo công thức của bài.
    experimental=float(joint.sum()/given_b.sum())  # Đếm hoặc cộng phần tử để tính tổng/tỷ lệ.
    return theoretical,experimental  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task04_bayes():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    p_spam=0.30  # Tính hoặc cập nhật biến p_spam theo công thức của bài.
    p_flag_given_spam=0.90  # Tính hoặc cập nhật biến p_flag_given_spam theo công thức của bài.
    p_flag_given_clean=0.05  # Tính hoặc cập nhật biến p_flag_given_clean theo công thức của bài.
    p_flag=p_spam*p_flag_given_spam+(1-p_spam)*p_flag_given_clean  # Tính hoặc cập nhật biến p_flag theo công thức của bài.
    posterior=p_spam*p_flag_given_spam/p_flag  # Tính hoặc cập nhật biến posterior theo công thức của bài.
    return p_flag,posterior  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task05_binomial():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    k=np.arange(0,9)  # Tính hoặc cập nhật biến k theo công thức của bài.
    probs=binom.pmf(k,n=8,p=0.75)  # Dùng hàm xác suất/kiểm định đúng tham số của đề.
    return k,probs,float(binom.pmf(6,n=8,p=0.75)),float(probs[6:].sum())  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def save_plot(k,probs):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    import matplotlib  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.
    matplotlib.use("Agg")  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
```

```
    import matplotlib.pyplot as plt  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.
    folder=Path(__file__).resolve().parent/"outputs"  # Tính hoặc cập nhật biến folder theo công thức của bài.
    folder.mkdir(exist_ok=True)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    fig,ax=plt.subplots()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    ax.bar(k,probs)  # Vẽ hoặc lưu biểu đồ`(plot: đồ`thị) để trình bày kết quả.
    ax.set(xlabel="Number of resolved requests (k)",ylabel="P(X=k)",title="Binomial(n=8, p=0.75)")  # Đặt nhãn, chú giải hoặc lưới cho hình vẽ.
    ax.set_xticks(k)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    ax.grid(True,axis="y")  # Đặt nhãn, chú giải hoặc lưới cho hình vẽ.
    target=folder/"binomial_distribution.png"  # Tính hoặc cập nhật biến target theo công thức của bài.
    fig.savefig(target,dpi=140,bbox_inches="tight")  # Vẽ hoặc lưu biểu đồ`(plot: đồ`thị) để trình bày kết quả.
    plt.close(fig)  # Đóng biểu đồ`sau khi dùng để giải phóng bộ nhớ.
    print("Saved:",target)  # In kết quả (output: kết quả) để người học đối chiếu.

def main():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    cards=task01_cards()  # Tính hoặc cập nhật biến cards theo công thức của bài.
    simulated=task02_free_throws()  # Tính hoặc cập nhật biến simulated theo công thức của bài.
    cond_theory,cond_exp=task03_conditional_days()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    p_flag,p_actual_spam=task04_bayes()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    k,prob,p_exact,p_at_least=task05_binomial()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    if not (cards["P(A)"]==0.5): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (cards["P(B)"]==0.4): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (cards["P(A and B)"]==0.2): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (cards["P(A or B)"]==0.7): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (cond_theory==1.0): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (abs(simulated["experimental_single"]-0.7)<0.04): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (np.isclose(p_flag,0.305)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (np.isclose(prob.sum(),1)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    print("Task 1:",cards)  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Task 2:",simulated)  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Task 3 P(A|B), theoretical/experimental:",cond_theory,cond_exp)  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Task 4 P(flag), P(spam|flag):",round(p_flag,5),round(p_actual_spam,5))  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Task 5 P(X=6), P(X>=6):",round(p_exact,6),round(p_at_least,6))  # In kết quả (output: kết quả) để người học đối chiếu.
    if "--no-plot" not in sys.argv:  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
        save_plot(k,prob)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
if __name__=="__main__":  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    main()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
```

4. Giải thích nội dung và cách giải

Câu 1

Tạo `set(range(1,11))`, lọc từng biến cố bằng set comprehension. Dùng `&` cho giao và `|` cho hợp, rồi chia số phần tử cho kích thước không gian mẫu 10.

Vì sao làm vậy? Mỗi thẻ 1–10 có cơ hội như nhau, nên xác suất là số thẻ hợp điều kiện chia cho tổng 10 thẻ. Hai biến cố cần được biểu diễn thành hai tập hợp để lấy giao (cả hai đúng) và hợp (ít nhất một đúng).

Quy trình: (1) Tạo 10 số bằng `set(range(1, 11))`. (2) Lọc số chẵn bằng `x % 2 == 0`, lọc số lớn hơn 6 bằng `x > 6`. (3) Đếm phần tử mỗi tập; dùng `&` lấy giao và `|` lấy hợp. (4) Chia cho `len(space)` để có xác suất, rồi kiểm tra.

Công thức Python:

```
space = set(range(1, 11))
A = {x for x in space if x % 2 == 0}
B = {x for x in space if x > 6}
p_a = len(A) / len(space)             # 0.5
p_b = len(B) / len(space)             # 0.4
```

```
p_both = len(A & B) / len(space)      # 0.2
p_either = len(A | B) / len(space)    # 0.7
```

Giải thích cú pháp: `range(1, 11)` chứa từ 1 đến 10, **không** lấy 11. `set` là tập hợp không chứa phần tử trùng, `%` là chia lấy dư, `==` so sánh bằng. Trong tập hợp, `&` là giao và `|` là hợp; `len` đếm số phần tử; `/` chia thực. `A | B` không phải chỉ những thẻ có đúng một trong hai tính chất: thẻ vừa chẵn vừa lớn hơn 6 vẫn được tính **một lần**.

Ví dụ và lỗi thường gặp: A gồm năm thẻ chẵn, B gồm bốn thẻ lớn hơn 6, giao gồm 8 và 10. P(A∩B)=2/10, P(A∪B)=7/10.

Thuật ngữ cần nhớ: Sample space (không gian mẫu).

Câu 2

`np.random.default_rng(SEED)` tạo bộ sinh số có thể lặp; mỗi lần so ngẫu nhiên với ngưỡng 0,7/0,6 được một kết quả True/False. `mean()` cho tần suất thành công; dùng `&` và `|` trên hai mảng bool.

Vì sao giải bằng hai cách? Lý thuyết cho giá trị chính xác từ xác suất đã biết; mô phỏng tạo nhiều phép thử ngẫu nhiên để đối chiếu xem tần suất có gần lý thuyết hay không. Không yêu cầu 5.000 lượt phải cho đúng bằng 0.7.

Quy trình: (1) Tạo bộ sinh ngẫu nhiên với seed cố định cho dễ tái lập. (2) Sinh 5.000 số trong khoảng từ 0 đến 1, so sánh với tỷ lệ trúng rổ. (3) Với hai người chơi độc lập, lấy `&` để đếm cùng trúng, `|` để đếm ít nhất một trúng. (4) Tính trung bình các giá trị đúng/sai, rồi so sánh với lý thuyết.

Công thức Python:

```
import numpy as np
rng = np.random.default_rng(2026)
first = rng.random(5000) < 0.7
second = rng.random(5000) < 0.6
p_sim_both = (first & second).mean()
p_sim_either = (first | second).mean()
p_exact_both = 0.7 * 0.6                # 0.42
p_exact_either = 1 - (1 - 0.7) * (1 - 0.6) # 0.88
```

Giải thích hàm và tham số: `default_rng(2026)` dùng hạt giống (seed) để cùng code tạo cùng chuỗi số; `random(5000)` tạo đúng 5.000 lượt; `<` xét thành công; `.mean()` trên mảng Boolean lấy tỷ lệ giá trị `True`. `&` yêu cầu cả hai cùng trúng; `|` chỉ cần một người trúng. Biểu thức `1 - ...` lấy phần bù của trường hợp **cả hai cùng trượt**, tránh đếm trùng.

Ví dụ và lỗi thường gặp: Tần suất trong 5.000 lượt rút là kết quả mô phỏng, có thể lệch lý thuyết. Với hai người độc lập, P(cả hai)=0,7×0,6=0,42.

Thuật ngữ cần nhớ: Simulation (mô phỏng).

Câu 3

Đặt thứ Hai đến Chủ nhật ứng với mã 0–6; B={0,1,2,3} là tập con của A={0..4}. Trong mô phỏng, đếm số lượt chọn vào B rồi kiểm tra đồng thời A∩B để được xác suất có điều kiện.

Vì sao dùng mẫu số là B? Đề đã cho điều kiện ngày thuộc B (thứ Hai–Năm); trong bốn ngày này, tất cả đều là ngày làm việc A (thứ Hai–Sáu). Vì vậy xác suất có điều kiện bằng 1. Không chia cho cả 7 ngày trong tuần.

Quy trình: (1) Biểu diễn bảy ngày thành số 0–6; A là 0–4, B là 0–3. (2) Lấy giao `A & B` để tìm ngày thuộc cả hai. (3) Đếm giao và đếm B để chia. (4) Nếu mô phỏng, chỉ giữ lượt rơi vào B rồi so sánh tỷ lệ.

Công thức Python:

```
A = set(range(5))      # Thứ Hai đến thứ Sáu
B = set(range(4))      # Thứ Hai đến thứ Năm
p_a_given_b = len(A & B) / len(B)  # 1.0
rng = np.random.default_rng(2027)
drawn = rng.integers(0, 7, size=5000)
```

```
mask_b = np.isin(drawn, list(B))
mask_both = np.isin(drawn, list(A & B))
p_sim = mask_both.sum() / mask_b.sum()
```

Giải thích hàm và tham số: `integers(0, 7, size=5000)` sinh số nguyên từ **0 đến 6** (`7` không được lấy), 5.000 lần. `np.isin` kiểm tra mỗi ngày có thuộc danh sách không, trả mảng Boolean; `.sum()` đếm các giá trị `True`. `size=5000` là số lượt mô phỏng; `len(B)` là số trường hợp của điều kiện đã biết.

Ví dụ và lỗi thường gặp: Biến cố B (thứ Hai–thứ Năm) nằm hoàn toàn trong biến cố A (thứ Hai–thứ Sáu), nên P(A|B)=1.

Thuật ngữ cần nhớ: Conditional probability (xác suất có điều kiện).

Câu 4

Tính tổng hai cách bị hệ thống gắn cờ spam: spam thật bị phát hiện và email sạch bị báo nhầm. Sau đó dùng Bayes đổi chiều điều kiện từ P(flag|spam) sang P(spam|flag).

Vì sao không dùng trực tiếp 90%? 90% là tỷ lệ bị gắn cờ **trong các thư spam**, còn đề hỏi thư đã bị gắn cờ có khả năng thật sự là spam bao nhiêu. Nhóm thư bị gắn cờ còn chứa cả thư sạch bị gắn nhầm. Cần tính cả hai nguồn gắn cờ trước khi chia.

Quy trình: (1) Đặt tỷ lệ spam và thư sạch. (2) Tính phần thư spam bị gắn cờ. (3) Tính phần thư sạch bị gắn cờ nhầm. (4) Cộng hai phần để có tổng xác suất gắn cờ. (5) Chia phần spam bị gắn cờ cho tổng đã tính.

Công thức Python:

```
p_spam = 0.30
p_flag_given_spam = 0.90
p_flag_given_clean = 0.05
p_clean = 1 - p_spam
p_flag = p_spam * p_flag_given_spam + p_clean * p_flag_given_clean # 0.305
p_spam_given_flag = (p_spam * p_flag_given_spam) / p_flag # 0.8852459...
```

Giải thích phép tính: `p_flag_given_spam` là xác suất gắn cờ khi đã biết thư là spam (độ nhạy); `p_flag_given_clean` là tỷ lệ gắn nhầm thư sạch. `*` kết hợp tỷ lệ nhóm với tỷ lệ gắn cờ trong nhóm, `+` cộng hai trường hợp không giao nhau, `/` chuyển sang xác suất **trong nhóm đã bị gắn cờ**. Kết quả khoảng **88.52%**, không phải 90%.

Ví dụ và lỗi thường gặp: P(flag)=0,3×0,9+0,7×0,05=0,305. P(spam|flag)=0,27/0,305, không đánh đồng với P(flag|spam)=0,9.

Thuật ngữ cần nhớ: Bayes (định lý Bayes).

Câu 5

Số thành công tuân theo nhị thức vì có 8 phép thử độc lập cùng xác suất p=0,75. Hàm pmf trả xác suất đúng k; cộng pmf cho k=6,7,8 để lấy ít nhất 6. Hàm `save_plot` trong code tham khảo dùng `plt.bar`.

Vì sao dùng phân phối nhị thức? Mỗi yêu cầu được xử lý thành công hoặc thất bại, có xác suất thành công 0.75 và giả định các lần độc lập. Bài cần xác suất đúng k thành công trong 8 lần, rồi cộng xác suất của k bằng 6, 7 hoặc 8.

Quy trình: (1) Tạo danh sách số lần thành công có thể từ 0 đến 8. (2) Dùng hàm phân phối nhị thức tính xác suất cho từng số k. (3) Lấy xác suất đúng 6 lần. (4) Cộng ba giá trị ứng với k từ 6 trở lên. (5) Vẽ biểu đồ cột để nhìn khả năng từng số lần thành công.

Công thức Python:

```
from scipy.stats import binom
import numpy as np
k = np.arange(0, 9)
probs = binom.pmf(k, n=8, p=0.75)
```

```
p_exactly_6 = binom.pmf(6, n=8, p=0.75)
p_at_least_6 = probs[6:].sum()
```

Giải thích hàm và tham số: `binom.pmf` là hàm khối xác suất (*probability mass function*) tính xác suất đúng k lần thành công; `n=8` là tổng số lượt, `p=0.75` là tỷ lệ thành công mỗi lượt. `np.arange(0, 9)` tạo các số 0–8; `probs[6:]` lấy các xác suất từ vị trí 6 đến hết; `.sum()` cộng chúng. Nếu vẽ và lưu ảnh, `matplotlib.use("Agg")` dùng backend không mở cửa sổ và `--no-plot` là đối số dòng lệnh bỏ phần vẽ trong code chính.

Ví dụ và lỗi thường gặp: Với tám yêu cầu độc lập, dùng phân phối nhị thức. Đúng sáu dùng pmf(6); ít nhất sáu cộng pmf(6)+pmf(7)+pmf(8).

Thuật ngữ cần nhớ: Binomial distribution (phân phối nhị thức).